

AI Agent — Enterprise Knowledge Assistant

Complete Roadmap: [Learning](#) + [Parallel Implementation](#) + [Mini-Projects](#) + [Data Sourcing](#)

This document is your single reference for building the project end-to-end. It combines what to learn with what to build at the same time, so you never learn a concept without immediately applying it in code. It also includes standalone mini-projects you can publish separately on GitHub, and a full guide on where to get real data to work with.

Total core build time	~60-90 focused hours
With impressive extras	~95-140 focused hours
Primary language	Python (FastAPI) — C#/.NET path also documented
Core stack	Azure OpenAI, Azure AI Search, Microsoft Agent Framework, Redis

How to use this roadmap

Each week below has two tracks that run side by side: **Learn** (the concept/theory to study) and **Build** (the exact piece of the project you implement using that concept, the same week — not later). By the end of Week 8 the learning and the implementation finish at the same time, and you have a deployed, working product.

8-Week Roadmap — Learning and Implementation in Parallel

Week	Learn (theory/skills)	Build (implement the same week)	Hrs
1	Python async basics, REST API concepts, how LLM chat completion calls work, system vs user prompts.	Set up FastAPI project. Build a working <i>/ask</i> endpoint that calls Azure OpenAI/OpenAI and returns a plain answer. No documents yet — this is your raw LLM baseline.	6-10
2	What embeddings are, cosine similarity, why 'search by meaning' beats keyword search.	Script that embeds 5-10 sample sentences + a question, and manually finds the closest match using numpy — no database yet, pure intuition-building.	5-8
3	PDF/Word/Excel text extraction, chunking strategy (overlap, chunk size trade-offs), metadata preservation.	Ingestion pipeline: any PDF/Word/Excel file in, clean JSON chunks out (with source file + page number attached to every chunk).	8-12
4	Vector database concepts, indexing, approximate nearest neighbor search, Azure AI Search basics.	Push your Week 3 chunks + embeddings into Azure AI Search (or ChromaDB locally first). Build the retrieval function: question in, top-5 relevant chunks out.	8-12
5	Prompt engineering for grounded answers, structured output (JSON), citation design.	Connect retrieval + LLM: real question in, real answer out, with exact source file + page cited. This is your first true end-to-end RAG pipeline.	8-12
6	Agent frameworks, function/tool calling, how an LLM decides which tool to invoke.	Wire up Microsoft Agent Framework. Add a second tool (SQL/SQLite query) alongside document search so the agent autonomously picks the right source.	10-14
7	Session/conversation memory, feedback loops for ML systems, role-based access control.	Add Redis-backed memory for follow-up questions, a thumbs up/down endpoint, and permission-filtered retrieval (users only see documents they're allowed to see).	8-12
8	Deployment basics, logging/observability, writing technical documentation.	Add a simple chat UI, deploy to a live URL, wire up Application Insights, write the README + blog post, record the demo video.	8-12

Core build subtotal: ~60-90 focused hours. Add the enhancement layer below (multi-format ingestion, confidence scoring, Slack/Teams bot, evaluation suite, analytics dashboard) for another ~35-50 hours to reach a genuinely portfolio-standout project.

Main Project to Publish on GitHub

Enterprise Knowledge Assistant (RAG Agent) — the full build from the roadmap above. This is your flagship repo. To make it stand out beyond a typical student RAG project, include these enhancements (each maps to a week above):

- **Precise citations** — link to exact page/paragraph, not just filename (Week 5)
- **Multi-format ingestion** — PDF, Word, Excel, and OCR for scanned images (Week 3 extension)
- **Access-control aware retrieval** — role-based filtering so users only see permitted docs (Week 7)
- **Feedback-driven re-ranking** — thumbs up/down improves future answer ranking (Week 7)
- **Confidence scoring** — agent flags low-confidence answers instead of guessing
- **Automated evaluation suite** — a fixed set of test Q&A; pairs run on every change to catch regressions
- **Usage analytics dashboard** — most-asked questions, response time, knowledge gaps

Repo checklist: working prototype with a live deployed link, clean README with setup steps, architecture diagram, a 3-5 minute demo video, and a short blog post explaining your design decisions.

Mini-Projects — One Per Concept, Each Its Own GitHub Repo

Publishing small, focused repos alongside your main project shows you understand each underlying concept individually, not just as part of one big app. Each of these takes a few hours and can be built the same week you learn the concept.

Mini-project	Concept it proves	What it does	Hrs
semantic-search-cli	Embeddings + cosine similarity (Week 2)	Command-line tool: paste in a list of sentences, type a query, it ranks all sentences by semantic closeness. No frameworks, just numpy.	3-5
doc-chunker	Ingestion + chunking strategy (Week 3)	Standalone package that takes any PDF/DOCX/XLSX and outputs clean, overlapping text chunks with page numbers as JSON. Reusable in other projects.	4-6
mini-rag-chatbot	End-to-end RAG without a full agent framework (Week 4-5)	A tiny Streamlit or CLI chatbot backed by ChromaDB/FAISS (no Azure needed) — upload a PDF, ask questions, get cited answers. Great 'proof of concept' repo.	6-8
tool-calling-agent-demo	Agent tool calling (Week 6)	A simple agent that autonomously chooses between a calculator tool, a weather API tool, and a SQLite lookup tool based on the question — isolates the 'autonomy' concept.	5-7
rbac-retrieval-demo	Access-control aware retrieval (Week 7)	Two demo users with different roles query the same document set and get different, correctly filtered results. Shows you understand enterprise security concerns.	4-6

Mini-project	Concept it proves	What it does	Hrs
rag-eval-harness	Evaluation and quality measurement	A small test harness that runs a fixed set of Q&A; pairs against your RAG pipeline and scores answer accuracy — the kind of tool real AI teams use before shipping.	5-7

Tip: pin these 6 mini-repos plus your main project on your GitHub profile. Together they tell a story — 'I understand each building block AND I can assemble them into a real product.'

How to Get Data for This Project

You don't have real company documents, and that's completely fine — here are practical, legitimate ways to get realistic data at every stage.

1. Your own documents (best starting point)

- Your resume, cover letters, old assignment reports, class notes, project proposals — anything you already have as PDF/Word/Excel.
- This is the fastest way to start Week 3 immediately without waiting on anything external.

2. Public datasets that resemble enterprise documents

- **SEC EDGAR** (sec.gov/edgar) — real company filings, 10-Ks, annual reports. Great stand-in for 'contracts and proposals'.
- **Kaggle** — search 'contracts dataset', 'invoice dataset', 'business documents dataset', 'email dataset (Enron)'.
- **Enron Email Dataset** — a famous, freely available real corporate email archive, perfect for simulating 'email threads'.
- **Annual reports / investor decks** — most public companies post these as free PDFs on their investor relations pages.
- **Government open-data portals** — data.gov, data.gov.uk — policy documents, technical reports.

3. Synthetic data generation (fastest, fully controllable)

- Use an LLM (ChatGPT/Claude/Azure OpenAI) to generate realistic fake company documents: sample contracts, pricing proposals, project retrospectives, meeting notes. Ask for 15-20 documents with varied topics and consistent fictional client/project names so your citation and retrieval demos look coherent.
- Generate a small 'client history' — e.g. 5 fictional clients, each with 3-4 documents (proposal, contract, follow-up email) — so your example query ('What pricing did we offer Client X last year?') actually has a real, correct answer to find.
- This also gives you full control to test edge cases: conflicting info across documents, outdated pricing, documents that should NOT be visible to certain roles (for your access-control feature).

4. Scanned/image documents (for your OCR feature)

- Print a few of your synthetic documents to PDF, then screenshot or photograph them to create 'scanned-looking' images — instant OCR test data.
- Kaggle also has scanned-invoice and scanned-form datasets if you want variety.

5. Slack/Teams-style data (for your chat export feature)

- Export your own real Slack/Discord conversations (with sensitive info removed), or generate synthetic back-and-forth conversations with an LLM in a realistic export format (JSON with timestamps, usernames, messages).

A word on privacy and licensing

Avoid uploading anyone else's private or copyrighted documents to a public GitHub repo. For public datasets, check the license before redistributing the raw files in your repo — usually it's fine to include a small sample subset and instructions for downloading the rest yourself. Synthetic data sidesteps this entirely, which is why it's the safest option for a portfolio project.

Good luck — build in order, ship something working every week, and don't wait for the project to be 'perfect' before you publish the first version to GitHub.